



PROJECT REPORT

E-Commerce Sales Analytics

A Python and pandas based console application for exploring sales data

Student Name	Bhavya Chauhan
Registration No.	26BAI10579
Project Name	E-Commerce Sales Analytics

Table of Contents

1. Cover Page
2. Introduction
3. Problem Statement
4. Functional Requirements
5. Non-functional Requirements
6. System Architecture
7. Design Diagrams
8. Design Decisions and Rationale
9. Implementation Details
10. Screenshots / Results
11. Testing Approach
12. Challenges Faced
13. Conclusion and Future Scope

2. Introduction

Online retail businesses generate a large number of transactions every day. Each order records what was sold, to whom, where, when and how it was paid for. On its own this raw data is hard to read; it becomes useful only when it is summarised into figures such as total revenue, best-selling products, strongest cities and preferred payment methods.

E-Commerce Sales Analytics is a menu-driven console application written in Python that uses the **pandas** library to turn order-level sales records into such summaries. The user chooses an option from a main menu and immediately sees a formatted report: key performance indicators (KPIs), sales and profit by category, top products, sales by city, monthly sales trend and payment method analysis. The user can also narrow the whole analysis to a particular **category** and/or **city** using an interactive filter, and reset it at any time.

The dataset contains ten orders placed between January and April 2026 across four cities of Uttar Pradesh (Lucknow, Kanpur, Agra and Varanasi) in three product categories (Electronics, Furniture and Fashion). The data is embedded in the program as a CSV string, so the project runs on any machine that has Python and pandas installed, with no external files or database to set up.

Objectives

- Load and prepare sales data using pandas (type conversion, parsing dates).
- Compute key business metrics using group-by aggregation.
- Provide an easy, interactive menu for non-technical users.
- Allow dynamic filtering by category and city so that every report can be viewed for a specific segment.
- Keep the code modular so that new reports or a real data source can be added later.

3. Problem Statement

A small e-commerce seller keeps its orders as plain rows of data. To understand business performance the owner has to manually add up columns, sort products and compare cities, which is slow, repetitive and error-prone. There is no quick way to answer questions such as *What is our total profit? Which category earns the most? Which city buys most? Which month was the best? Do customers prefer UPI or cards?*

The problem is to design and implement a simple, reliable software tool that reads the sales data and answers these questions instantly through an interactive interface, while also letting the user restrict the analysis to a chosen category and city, and displaying the results in a clean, readable format.

Scope

In scope	Out of scope
Console-based interface; static embedded dataset; descriptive analytics (sums, counts, rankings); category and city filters.	Graphical dashboards or charts; live database or file upload; predictive analytics; user login and multi-user access.

4. Functional Requirements

The functional requirements describe what the system must do. Each requirement maps to a menu option or function in the code.

ID	Requirement	Implemented by
FR1	The system shall load the sales dataset and convert Order_Date to a date type at start-up.	pd.read_csv, pd.to_datetime
FR2	The system shall display a main menu with numbered options and repeat it until the user exits.	main()
FR3	The system shall show KPIs: total sales, total profit, total orders and total unique customers (Menu 1).	show_summary()
FR4	The system shall show total sales for each product category in descending order (Menu 2).	sales_by_category()
FR5	The system shall list the top 10 products ranked by sales (Menu 3).	top_products()
FR6	The system shall show total sales for each city in descending order (Menu 4).	sales_by_city()
FR7	The system shall show total sales for each calendar month (Menu 5).	monthly_sales()
FR8	The system shall show sales by payment method (Menu 6).	payment_analysis()
FR9	The system shall show total profit by category (Menu 7).	profit_by_category()
FR10	The system shall display the complete (or filtered) order table (Menu 8).	show_data()
FR11	The system shall let the user filter data by category and by city, and report the number of matching records (Menu 9).	filter_data()
FR12	The system shall reset all filters to the full dataset (Menu 10).	df.copy()
FR13	The system shall apply the active filter to every report (Menus 1-8).	filtered_df argument
FR14	The system shall handle invalid menu input by showing a message and re-displaying the menu.	else branch in main()
FR15	The system shall exit cleanly and print the project credit on termination (Menu 0).	main(), credit block

5. Non-functional Requirements

Attribute	Requirement	How the project meets it
Usability	Menu must be simple enough for a non-programmer.	Numbered menu, section banners, clear prompts and messages.
Performance	Every report should appear instantly for the given data size.	Vectorised pandas operations (groupby/sum); no loops over rows.
Reliability	Wrong input must not crash the program.	Input checked with isdigit() and range checks; unknown options return to the menu.
Maintainability	Code should be easy to read and extend.	One function per report; consistent naming; comments separating sections.
Portability	Should run on Windows, Linux and macOS.	Pure Python 3 with pandas; no OS-specific code or external files.
Accuracy	Numbers must be correct and formatted consistently.	Aggregation done by pandas; currency shown with thousands separators and two decimals.
Data integrity	Original data must not be altered by filtering.	Filters work on copies (df.copy()); the global df is never modified.
Extensibility	New reports or data sources can be added.	Analysis functions accept any DataFrame, so a database or CSV file can replace the embedded data.

6. System Architecture

The application follows a simple **layered architecture** inside a single Python script. Each layer has one responsibility and talks only to the layer next to it, which keeps the code easy to understand and test.

Layer	Responsibility	Code elements
Presentation layer	Shows the menu, reads the user's choice, calls the right function and controls the program loop.	main(), input(), print()
Filter module	Interactively narrows the data by category and city and returns a new DataFrame.	filter_data()
Analytics module	Performs aggregations (sum, count, unique, sort) on whichever DataFrame it receives.	show_summary(), sales_by_category(), top_products(), sales_by_city(), monthly_sales(), payment_analysis(), profit_by_category(), show_data()
Data layer	Holds the raw data and prepares it for analysis.	data (CSV string), StringIO, pd.read_csv(), pd.to_datetime(), global df
Output layer	Formats results for the console.	f-strings, to_string(), banner lines of '=' characters

Data flow. The CSV text is converted to a DataFrame *df* once when the program starts. *main()* keeps a second variable, *filtered_df*, which starts as a copy of *df*. All reports receive *filtered_df*. When the user chooses Filter Data, *filter_data(df)* returns a new filtered DataFrame that replaces

filtered_df; choosing Reset Filters sets it back to a fresh copy of *df*.

Technology stack. Python 3, pandas (data handling), io.StringIO (treats the embedded text as a file). No database and no third-party interface libraries are needed.

7. Design Diagrams

7.1 Use Case Diagram

The only actor is the user (analyst / shop owner). Every menu option is a use case inside the system boundary.

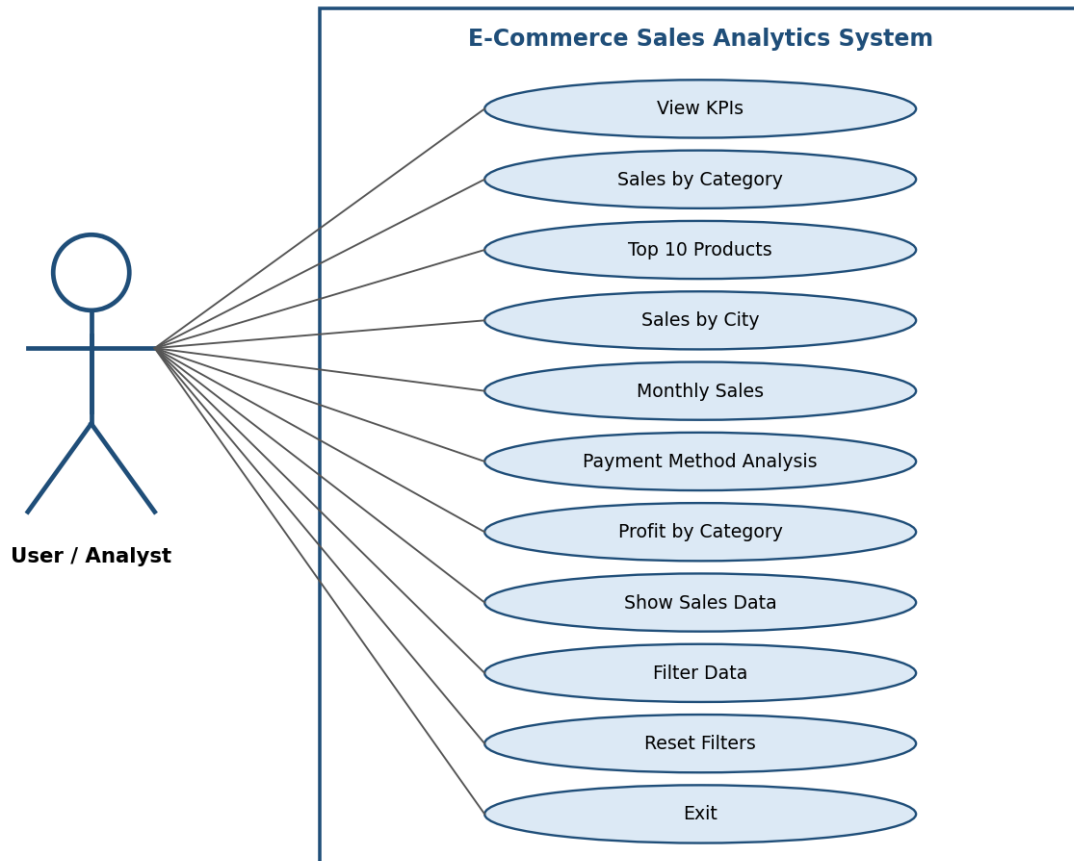


Figure 1: Use case diagram

7.2 Workflow Diagram

The workflow shows the program from start-up through the menu loop to exit.

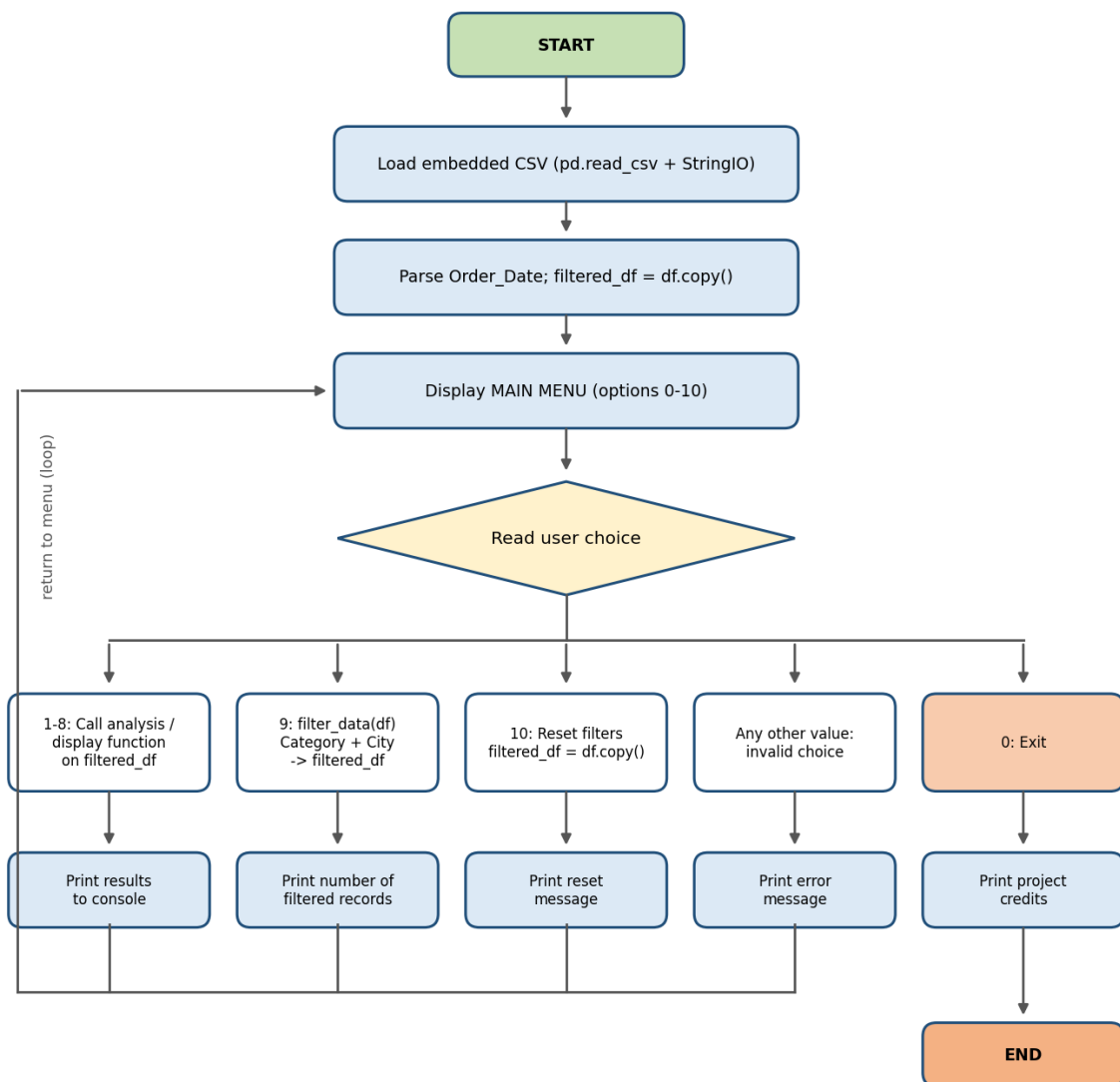


Figure 2: Workflow (flowchart) of the application

7.3 Sequence Diagram

The sequence below shows a typical session: the user applies a filter (option 9) and then views a report (option 2) on the filtered data.

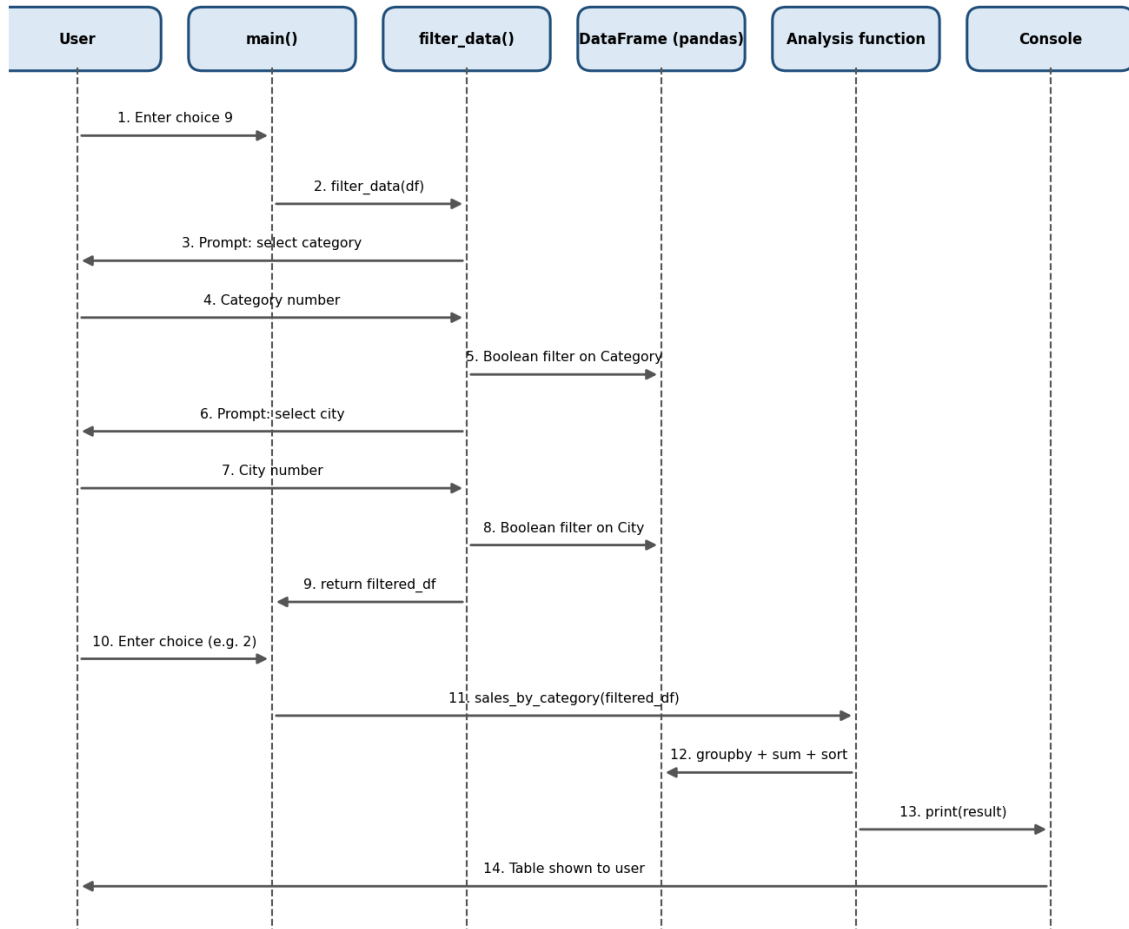


Figure 3: Sequence diagram - filtering and reporting

7.4 Class / Component Diagram

The project is written in a procedural style, so there are no classes; the design is shown as a component diagram of layers and their functions.

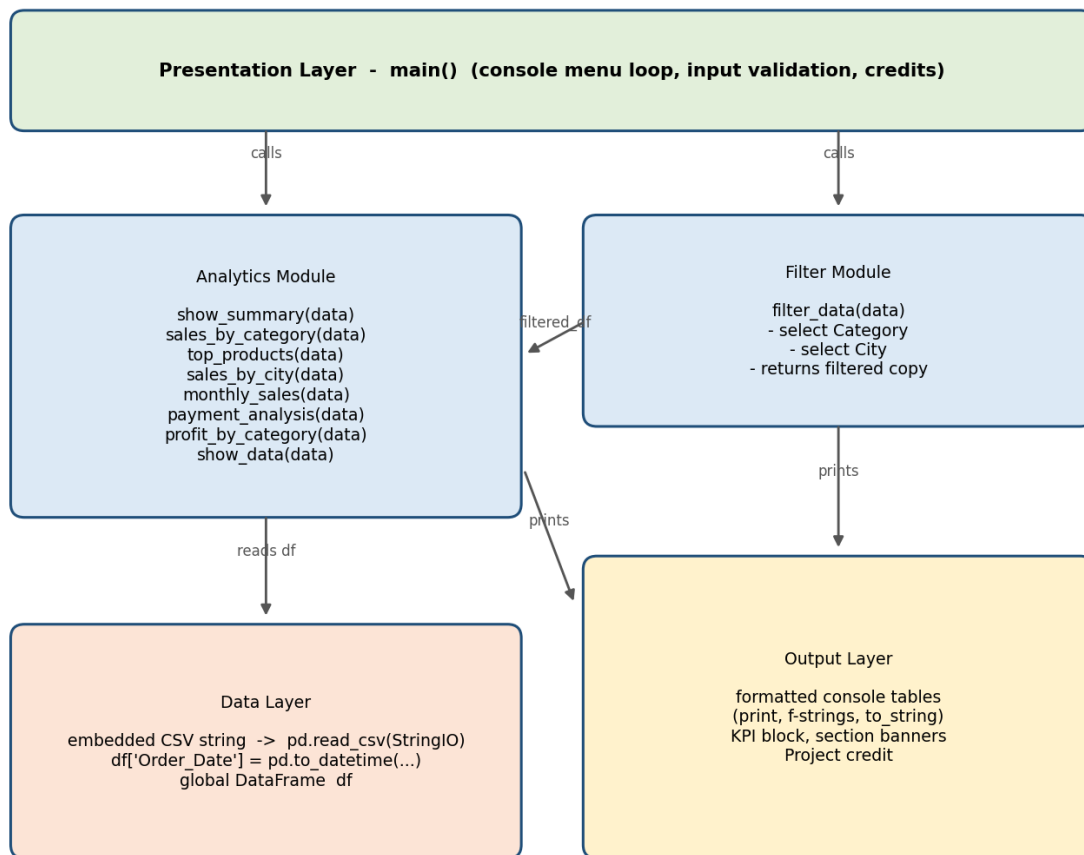


Figure 4: Component diagram

7.5 ER Diagram

The project does **not** use a database; the data lives in memory in a single pandas DataFrame. The ER view below documents that one entity so the data structure is clear. If the project is later moved to a database, Customer, Product and Order could be split into separate tables.

SALES_ORDERS (DataFrame df)	
Order_ID	INT (PK)
Order_Date	DATE
Customer_ID	STRING
Product	STRING
Category	STRING
City	STRING
Sales	NUMBER
Profit	NUMBER
Payment_Method	STRING

Figure 5: Data model of the SALES_ORDERS DataFrame

8. Design Decisions and Rationale

Decision	Rationale
Embed the dataset as a CSV string and read it with StringIO	The program is self-contained: it runs anywhere without extra files, which is ideal for submission and demonstration.
Use pandas for all calculations	groupby, sum, nunique and sort_values give short, readable and fast code compared with manual loops.
Console menu instead of a GUI	Simple to build and test, works over any terminal, and keeps the focus on data analysis rather than interface design.
One function per report, each taking a DataFrame parameter	The same function works for full and filtered data; new reports can be added without touching existing ones.
Keep df (original) and filtered_df (working copy) separate	Filtering never damages the source data and Reset Filters is a one-line copy.
Filters are applied sequentially (category, then city) and return a new DataFrame	Easy for users to follow and produces predictable results; choosing 0 keeps that dimension unfiltered.
Group by month using dt.to_period('M')	Produces clean YYYY-MM labels and correct chronological order for the trend.
Validate menu and filter input with isdigit() and range checks	Prevents crashes from letters, blanks or out-of-range numbers.
Format money with f'{value:,.2f}' and the rupee sign	Matches the Indian retail context and improves readability of large figures.
Print project credit at the end	Identifies the author of the work as required for submission.

9. Implementation Details

9.1 Data loading and preparation

The CSV text is read into a DataFrame with `pd.read_csv(StringIO(data))`. The `Order_Date` column is converted with `pd.to_datetime` so that month-wise grouping works. The dataset has nine columns: `Order_ID`, `Order_Date`, `Customer_ID`, `Product`, `Category`, `City`, `Sales`, `Profit` and `Payment_Method`, and ten records.

9.2 Analysis functions

Function	Logic	pandas operation
show_summary	Total sales and profit; number of unique orders and customers.	sum(), nunique()
sales_by_category	Group by Category, add Sales, sort high to low.	groupby().sum().sort_values()
top_products	Group by Product, add Sales, keep first ten.	groupby().sum().sort_values().head(10)
sales_by_city	Group by City, add Sales, sort.	groupby().sum().sort_values()
monthly_sales	Group by month period of Order_Date.	dt.to_period('M')
payment_analysis	Group by Payment_Method, add Sales, sort.	groupby().sum().sort_values()
profit_by_category	Group by Category, add Profit, sort.	groupby().sum().sort_values()
show_data	Print the whole table without the index.	to_string(index=False)

9.3 Filtering

filter_data lists the distinct categories (sorted) with numbers and adds option 0 for 'All'. The user's number is validated and, if valid, the DataFrame is reduced with a boolean condition such as `filtered_data[filtered_data['Category'] == selected_category]`. The same is repeated for City. A choice of 0, a non-numeric value or an out-of-range number leaves that dimension unfiltered. The function returns the resulting DataFrame.

9.4 Main program loop

main() prints the banner and record count, then runs a `while True` loop that shows the menu, reads the choice and dispatches to the matching function through an `if/elif` chain. Option 0 breaks the loop. The script is started with the standard `if __name__ == '__main__':` guard.

9.5 How to run

- Install Python 3 and pandas (`pip install pandas`).
- Save the code as `sales_analytics.py`.
- Run `python sales_analytics.py` in a terminal and choose options from the menu.

10. Screenshots / Results

The following console outputs were produced by running the project's functions on the embedded dataset. Replace or add your own terminal screenshots here if your course requires image screenshots.

10.1 Data preparation check

```
$ python sales_analytics.py | startup date check

CHECKING ORDER DATES:
0  2026-01-05
1  2026-01-08
2  2026-01-12
3  2026-02-03
4  2026-02-10
5  2026-02-15
6  2026-03-02
7  2026-03-11
8  2026-03-18
9  2026-04-04
Name: Order_Date, dtype: datetime64[us]
```

10.2 Key performance indicators (Menu 1)

```
$ python sales_analytics.py | menu 1
```

```
=====
KEY PERFORMANCE INDICATORS
=====
Total Sales      : ₹195,200.00
Total Profit     : ₹29,700.00
Total Orders     : 10
Total Customers  : 10
```

10.3 Sales by category and profit by category (Menus 2 and 7)

```
$ python sales_analytics.py | menu 2
```

```
=====
SALES BY CATEGORY
=====
Category
Electronics    163000
Furniture      20500
Fashion        11700
```

```
$ python sales_analytics.py | menu 7
```

```
=====
PROFIT BY CATEGORY
=====
Category
Electronics    23300
Furniture      4000
Fashion        2400
```

10.4 Top products (Menu 3)

```
$ python sales_analytics.py | menu 3

=====
TOP 10 PRODUCTS
=====
Product
Laptop          55000
Smart TV        42000
Refrigerator    38000
Mobile Phone    25000
Table           12000
Office Chair    8500
Watch           6000
Shoes           4500
Headphones      3000
T-Shirt         1200
```

10.5 Sales by city (Menu 4)

```
$ python sales_analytics.py | menu 4

=====
SALES BY CITY
=====
City
Lucknow         69500
Kanpur          67500
Varanasi        54000
Agra            4200
```

10.6 Monthly sales (Menu 5)

```
$ python sales_analytics.py | menu 5

=====
MONTHLY SALES
=====
Order_Date
2026-01         83000
2026-02         25000
2026-03         49200
2026-04         38000
Freq: M
```

10.7 Payment method analysis (Menu 6)

```
$ python sales_analytics.py | menu 6

=====
PAYMENT METHOD ANALYSIS
=====
Payment_Method
UPI             104500
Credit Card    81000
Cash            9700
```

10.8 Full data (Menu 8)

```
$ python sales_analytics.py | menu 8
```

```
=====
SALES DATA
=====
Order_ID Order_Date Customer_ID Product Category City Sales Profit Payment_Method
1001 2026-01-05 C001 Laptop Electronics Lucknow 55000 7000 UPI
1002 2026-01-08 C002 Mobile Phone Electronics Kanpur 25000 4000 Credit Card
1003 2026-01-12 C003 Headphones Electronics Agra 3000 800 UPI
1004 2026-02-03 C004 Office Chair Furniture Lucknow 8500 1500 Cash
1005 2026-02-10 C005 Table Furniture Varanasi 12000 2500 Credit Card
1006 2026-02-15 C006 Shoes Fashion Kanpur 4500 900 UPI
1007 2026-03-02 C007 T-Shirt Fashion Agra 1200 300 Cash
1008 2026-03-11 C008 Watch Fashion Lucknow 6000 1200 Credit Card
1009 2026-03-18 C009 Smart TV Electronics Varanasi 42000 6000 UPI
1010 2026-04-04 C010 Refrigerator Electronics Kanpur 38000 5500 Credit Card
```

10.9 Filtering - Electronics in Kanpur (Menu 9, then Menu 8)

```
$ python sales_analytics.py | menu 9
```

```
=====
FILTER DATA
=====

Categories:
1. Electronics
2. Fashion
3. Furniture
0. All Categories
```

```
Select category: 1
```

```
Cities:
1. Agra
2. Kanpur
3. Lucknow
4. Varanasi
0. All Cities
```

```
Select city: 2
```

```
$ python sales_analytics.py | menu 8 after filter
```

```
=====
SALES DATA
=====
Order_ID Order_Date Customer_ID Product Category City Sales Profit Payment_Method
1002 2026-01-08 C002 Mobile Phone Electronics Kanpur 25000 4000 Credit Card
1010 2026-04-04 C010 Refrigerator Electronics Kanpur 38000 5500 Credit Card
```

10.10 Key findings

- Total sales are **Rs. 1,95,200** with a total profit of **Rs. 29,700** (about 15.2% profit margin) from 10 orders by 10 customers.
- **Electronics** dominates: Rs. 1,63,000 in sales (about 83.5% of the total) and Rs. 23,300 in profit.
- **Lucknow** (Rs. 69,500) and **Kanpur** (Rs. 67,500) are the top cities; Agra contributes the least (Rs. 4,200).
- **January** was the strongest month (Rs. 83,000), driven by the laptop sale.

- **UPI** is the most used payment method by value (Rs. 1,04,500), followed by credit card and cash.

11. Testing Approach

Because the program is interactive and small, testing was done manually (black-box) by running each menu option, comparing results with hand calculations and trying invalid inputs. The analysis functions take a DataFrame as an argument, so they can also be tested automatically with unit tests.

#	Test case	Input	Expected result	Result
1	KPI totals	Menu 1	Sales 1,95,200; Profit 29,700; 10 orders; 10 customers	Pass
2	Category ranking	Menu 2	Electronics 1,63,000 > Furniture 20,500 > Fashion 11,700	Pass
3	Top products	Menu 3	Laptop first, T-Shirt last (10 products)	Pass
4	City totals	Menu 4	Lucknow 69,500; Kanpur 67,500; Varanasi 54,000; Agra 4,200	Pass
5	Monthly grouping	Menu 5	Four months Jan-Apr 2026 with totals 83,000 / 25,000 / 49,200 / 38,000	Pass
6	Filter Electronics + Kanpur	9, then 1, then 2	Only orders 1002 and 1010 remain (2 records)	Pass
7	Report on filtered data	After test 6 choose Menu 1	Sales 63,000; Profit 9,500; 2 orders	Pass
8	Reset filters	Menu 10	All 10 records available again	Pass
9	Invalid menu choice	Input 'abc' or 99	'Invalid choice' message; menu shown again	Pass
10	Invalid filter input	Input 'x' or 9 in filter	No filter applied for that dimension; no crash	Pass
11	Exit	Menu 0	Thank-you message, loop ends, credit printed	Pass

Verification of calculations. Totals were cross-checked by adding the Sales and Profit columns by hand (for example $55,000 + 25,000 + 3,000 + 42,000 + 38,000 = 1,63,000$ for Electronics). The filtered result in test 7 follows from $25,000 + 38,000 = 63,000$ sales and $4,000 + 5,500 = 9,500$ profit.

Suggested automated tests. Using *pytest*, each function can be called with a small DataFrame and the expected aggregate compared with the returned value. Input-driven functions such as `filter_data` can be tested by mocking `input()`.

12. Challenges Faced

Challenge	How it was handled
Working with dates: month-wise totals need a real date type, not text.	Converted Order_Date with <code>pd.to_datetime</code> and grouped with <code>dt.to_period('M')</code> ; printed the date column at start-up to verify.
Applying filters across all reports without changing the original data.	Introduced separate <code>df</code> and <code>filtered_df</code> variables and passed the working copy to every function.
Handling wrong input in a menu program.	Used <code>isdigit()</code> and range checks so that letters, blanks and out-of-range numbers are ignored safely.
Presenting pandas output neatly in a console.	Used banner lines, f-string formatting and <code>to_string()</code> to get aligned, readable tables.
Formatting large amounts of money.	Applied thousands separators and two decimals with the rupee symbol.
Making the project run anywhere without data files.	Embedded the CSV inside the script and read it through <code>StringIO</code> .
Keeping repeated print code tidy.	Grouped each report into its own function with the same layout so it is easy to maintain.

13. Conclusion and Future Scope

The E-Commerce Sales Analytics project shows how a small pandas program can convert raw order records into useful business insight. It meets all the stated functional requirements, offers an easy menu-driven interface, supports category and city filtering, and keeps the code modular and readable.

Possible improvements

- Load data from a CSV/Excel file or a database (SQLite/MySQL) instead of the embedded string.
- Add charts using matplotlib or a web dashboard using Streamlit.
- Add date-range filters and filters for payment method.
- Export reports to CSV, Excel or PDF.
- Add profit-margin, average order value and repeat-customer metrics.
- Add automated unit tests and input handling for text with mixed case or spaces.

Project credit: Bhavya Chauhan | Registration No. 26BAI10579 | E-Commerce Sales Analytics